PropertyPanel - プロパティ編集パネル

1. 概要

1.1 コンポーネントの目的

PropertyPanelは、Petri Netエディタでノードまたはエッジが選択された際に表示される右側固定パネルです。選択された要素（ノード/エッジ）のプロパティを編集し、テンプレート管理、削除などの操作を提供します。

主要機能:

- ノードプロパティ編集（ID、名前、説明）
- エッジプロパティ編集（ID）
- テンプレート編集ダイアログ起動
- テンプレート継承設定
- 共通テンプレート設定・参照
- ノード・エッジ削除（確認モーダル付き）
- バリデーションエラー表示

1.2 使用場面

- ノードをクリックして選択した時
- エッジをクリックして選択した時
- React Flowキャンバスの右側に固定表示

1.3 アクセス権限

- **必須ロール**: Experiment Admin
- **認証**: JWT + Session Hybrid認証

---

2. 画面構成

2.1 レイアウト構造

```
┌──────────────────────────────────┐
│ ヘッダー（青色）                 │
│ [Place/Transition/Arc プロパティ]│
│                          [閉じる]│
├──────────────────────────────────┤
│                                  │
│ [エラー警告（エラーがある場合）] │
│                                  │
│ [要素タイプ表示カード]           │
│                                  │
│ [ID入力フィールド]               │
│                                  │
│ [名前入力フィールド]（ノードのみ）│
│                                  │
│ [説明入力エリア]（ノードのみ）   │
│                                  │
│ [共通テンプレートチェック]       │
│   （Placeノードのみ）            │
│                                  │
│ [テンプレート継承選択]           │
│   （Placeノードのみ）            │
│                                  │
│ [共通テンプレート参照]           │
│   （Placeノードのみ）            │
│                                  │
│ [テンプレート情報カード]         │
│   - 種類                         │
│   - プロパティ数                 │
│                                  │
│ [テンプレート編集ボタン]         │
│   （Placeノードのみ）            │
│                                  │
│ [削除ボタン]                     │
│                                  │
└──────────────────────────────────┘
```

2.2 主要UI要素

2.2.1 ヘッダー

- **背景色**: #1890ff（青色）
- **タイトル**:

  - Placeノード: "Place プロパティ"
  - Transitionノード: "Transition プロパティ"
  - エッジ: "Arc プロパティ"

- **閉じるボタン**: CloseOutlinedアイコン

2.2.2 エラー警告カード（エラーがある場合のみ表示）

- **背景色**: #fff3e0（オレンジ系）
- **ボーダー**: 1px solid #ff9800
- **内容**:
  - エラー件数表示
  - 各エラーの詳細（type、message、details）

2.2.3 要素タイプ表示カード

- **背景色**: #f5f5f5
- **内容**:
  - Placeノード: "Material Place" / "Condition Place" / "Result Place"
  - Transitionノード: "Transition"
  - エッジ: "Arc (エッジ)"

2.2.4 ID入力フィールド

- **ラベル**: "ノードID" / "Arc ID"
- **バリデーション**:
  - 空値チェック
  - 重複チェック（他のノードと重複不可）
- **エラー表示**: message.error

2.2.5 名前入力フィールド（ノードのみ）

- **ラベル**: "名前"
- **プレースホルダー**: "${placeType} の名前を入力"

2.2.6 説明入力エリア（ノードのみ）

- **ラベル**: "説明"
- **タイプ**: TextArea（4行）
- **プレースホルダー**: "ノードの説明を入力"

2.2.7 共通テンプレートチェック（Placeのみ）

- **ラベル**: "共通テンプレート（全methodで共有）"
- **説明**: "チェックすると、このテンプレートを他のmethodからも参照できます"

2.2.8 テンプレート継承選択（Placeのみ）

- **ラベル**: "テンプレート継承"
- **タイプ**: Select
- **オプション**:
  - "なし（独立テンプレート）"
  - 同一method内の同じplaceTypeのノード一覧
- **説明**: "同一method内のテンプレートからプロパティを継承"

2.2.9 共通テンプレート参照（Placeのみ、複数method時のみ表示）

- **ラベル**: "共通テンプレート参照"
- **タイプ**: Select
- **オプション**:
  - "なし"
  - 全methodの共通テンプレート一覧（"テンプレート名 (from Method名)"）
- **説明**: "共通テンプレートへのplaceRef参照を追加"

2.2.10 テンプレート情報カード（Placeのみ）

- **背景色**: #fafafa
- **内容**:
  - 種類: "Material Template" / "Condition Template" / "Result Template"
  - プロパティ数: "X件"

2.2.11 テンプレート編集ボタン（Placeのみ）

- **ラベル**: "テンプレート編集"
- **説明**: "プロパティの追加・編集・削除"

2.2.12 削除ボタン

- **ラベル**: "ノードを削除" / "エッジを削除"
- **タイプ**: danger（赤色）
- **アイコン**: DeleteOutlined
- **説明**: "このノード/エッジを完全に削除します"

---

3. 機能仕様

3.1 Props定義

```
export interface PropertyPanelProps {
  selectedNode: Node<NodeData> | null;
  selectedEdge: Edge<EdgeData> | null;
  onUpdate: (nodeId: string, data: NodeData) => void;
  onUpdateEdge?: (oldEdgeId: string, newEdgeId: string) => void;
  onClose: () => void;
  onOpenTemplateEditor: (node: Node<NodeData>) => void;
  onDelete: (nodeId: string) => void;
  onDeleteEdge?: (edgeId: string) => void;
  onUpdateNodeId?: (oldId: string, newId: string) => void;
  nodes: Node<NodeData>[];
```

```
  edges: Edge<EdgeData>[];
  methods: MethodData[];
  activeMethodIndex: number;
}
```

Props説明:

Prop名	型	必須	説明
selectedNode	Node | null	✅	選択中のノード
selectedEdge	Edge | null	✅	選択中のエッジ
onUpdate	(nodeId, data) => void	✅	ノードプロパティ更新コールバック
onUpdateEdge	(oldId, newId) => void	-	エッジID更新コールバック
onClose	() => void	✅	パネルを閉じるコールバック
onOpenTemplateEditor	(node) => void	✅	テンプレート編集ダイアログを開くコールバック
onDelete	(nodeId) => void	✅	ノード削除コールバック
onDeleteEdge	(edgeId) => void	-	エッジ削除コールバック
onUpdateNodeId	(oldId, newId) => void	-	ノードID変更コールバック
nodes	Node[]	✅	全ノード配列（重複チェック、テンプレート継承用）
edges	Edge[]	✅	全エッジ配列
methods	MethodData[]	✅	全メソッド配列（共通テンプレート検索用）
activeMethodIndex	number	✅	アクティブなメソッドのインデックス

3.2 State定義

```
const [deleteConfirmOpen, setDeleteConfirmOpen] = useState(false);
const [deleteEdgeConfirmOpen, setDeleteEdgeConfirmOpen] = useState(false);
```

State説明:

State名	型	初期値	説明
deleteConfirmOpen	boolean	false	ノード削除確認モーダルの開閉状態
deleteEdgeConfirmOpen	boolean	false	エッジ削除確認モーダルの開閉状態

3.3 計算値（useMemo）

3.3.1 availableTemplatesForInheritance

```
const availableTemplatesForInheritance = useMemo(() => {
  if (!selectedNode || !isPlace || !nodes) return [];

  const templates = nodes
    .filter(node =>
      node.data.nodeType === 'place' &&
      node.data.placeType === placeType &&
      node.id !== selectedNode.id
    )
    .map(node => ({
      id: node.id,
      name: node.data.name || node.id,
    }));

  return templates;
}, [nodes, isPlace, placeType, selectedNode]);
```

説明: 同一method内の同じplaceTypeのノード一覧（テンプレート継承用）

3.3.2 availableCommonTemplates

```
const availableCommonTemplates = useMemo(() => {
  if (!selectedNode || !isPlace || !methods || methods.length <= 1) return [];

  const commonTemplates: Array<{
    id: string;
    name: string;
    methodIndex: number;
    methodName: string;
  }> = [];

  methods.forEach((method, index) => {
    if (!method.nodes) return;

    const templates = method.nodes
      .filter(node =>
        node.data.nodeType === 'place' &&
        node.data.placeType === placeType &&
```

```
        node.data.isCommonTemplate === true &&
        node.id !== selectedNode.id
      )
      .map(node => ({
        id: node.id,
        name: node.data.name || node.id,
        methodIndex: index,
        methodName: method.name || `Method ${index + 1}`,
      }));

    commonTemplates.push(...templates);
  });

  return commonTemplates;
}, [methods, isPlace, placeType, selectedNode]);
```

説明: 全methodから共通テンプレート（isCommonTemplate=true）を抽出

3.4 プロパティ変更

3.4.1 処理フロー

```
const handleChange = (field: string, value: unknown) => {
  if (selectedNode) {
    onUpdate(selectedNode.id, {
      ...selectedNode.data,
      [field]: value,
    });
  }
};
```

3.4.2 UI動作

- ユーザー操作: 入力フィールドの値を変更
- 結果: 親コンポーネント（PetriNetEditorInner）のhandleUpdateNodeProperty実行
- 画面更新: 変更が即座に反映される

3.5 ノードID変更

3.5.1 処理フロー

```
onChange={(e) => {
  const newId = e.target.value;

  // 空値チェック
  if (!newId.trim()) {
    message.error('ノードIDは空にできません');
    return;
  }

  // 重複チェック
  const isDuplicate = nodes.some(n => n.id === newId && n.id !== selectedNode.id);
  if (isDuplicate) {
    message.error(`ID "${newId}" は既に使用されています`);
    return;
  }

  if (onUpdateNodeId) {
    onUpdateNodeId(selectedNode.id, newId);
  }
}}
```

3.5.2 UI動作

- ユーザー操作: ノードIDフィールドを編集
- バリデーション:
  1. 空値チェック → エラーメッセージ表示
  2. 重複チェック → エラーメッセージ表示
- 成功: 親コンポーネントのhandleUpdateNodeId実行
- 結果: ノードIDが変更され、接続エッジのsource/targetも自動更新

3.6 テンプレート継承設定

3.6.1 処理フロー

```
const handleTemplateRefChange = (href: string) => {
  if (!selectedNode) return;

  const updatedData: NodeData = {
    ...selectedNode.data,
    templateRef: href ? {
      id: href,
      category: placeType,
      uuid: '',
      name: '',
      placeRefs: [],
      templateRef: {id :'', ref: href},
      properties: [],
      contents: []
    } : undefined,
  };

  if (updatedData.template) {
    updatedData.template = {
      ...updatedData.template,
      templateRef: {id :'', ref: href},
    };
  }

  onUpdate(selectedNode.id, updatedData);
};
```

3.6.2 UI動作

- ユーザー操作: テンプレート継承Selectで他のノードを選択
- 結果: selectedNode.data.templateRefが設定される
- 効果: 選択したノードのテンプレートプロパティを継承

3.7 テンプレート編集ダイアログ起動

3.7.1 処理フロー

```
onClick={() => {
  // TemplateContextからテンプレートを復元
  let latestTemplate = null;
  const nodeId = selectedNode.id;
```

```
  const category = placeType as 'material' | 'condition' | 'result';

  if (allTemplates && allTemplates[category]) {
    latestTemplate = allTemplates[category].find((t: Template) =>
      t.placeRefs && t.placeRefs.some((ref: Reference) => ref.ref === nodeId)
    );
  }

  const updatedNode: Node<NodeData> = {
    ...selectedNode,
    data: {
      ...selectedNode.data,
    }
  };

  // 復元されたテンプレートを使用
  if (latestTemplate) {
    updatedNode.data.template = { ...latestTemplate };
    updatedNode.data.templateRef = { ...latestTemplate };
  }

  // 新規テンプレートの場合は初期データを設定
  if (!updatedNode.data.template || !updatedNode.data.template.id) {
    const templateId = `${placeType}Template_${Date.now()}`;
    updatedNode.data.template = {
      id: templateId,
      category: placeType,
      uuid: crypto.randomUUID(),
      name: selectedNode.data.name || '',
      description: selectedNode.data.description || '',
      placeRefs: [{ id: '', ref: nodeId }],
      properties: [],
      contents: []
    };
    updatedNode.data.templateRef = { ...updatedNode.data.template };
    updatedNode.data.isNewTemplate = true;
    updatedNode.data.templateCategory = placeType;
    updatedNode.data.isCommonTemplate = selectedNode.data.isCommonTemplate || false;
    updatedNode.data.methodId = !selectedNode.data.isCommonTemplate && methods &&
methods[activeMethodIndex]
      ? methods[activeMethodIndex].methodId
      : null;
  }

  onOpenTemplateEditor(updatedNode);
}}
```

3.7.2 UI動作

- ユーザー操作: "テンプレート編集"ボタンをクリック
- 処理:
  1. TemplateContextから最新のテンプレート情報を取得
  2. 既存テンプレートがあればそれを使用
  3. なければ新規テンプレートとして初期データを生成
  4. UUID、PlaceRefを設定
- 結果: TemplateEditorDialogが開く

3.8 ノード削除

3.8.1 処理フロー

```
// 削除ボタンクリック
onClick={() => setDeleteConfirmOpen(true)}

// 削除確認モーダルのOKボタン
const handleDeleteConfirm = () => {
  if (selectedNode && onDelete) {
    onDelete(selectedNode.id);
  }
  setDeleteConfirmOpen(false);
  onClose();
};
```

3.8.2 UI動作

- ユーザー操作: "ノードを削除"ボタンをクリック
- モーダル表示: "ノード削除の確認"
- 確認: "削除"ボタンをクリック
- 結果:
  1. 親コンポーネントのhandleDeleteNode実行
  2. ノードが削除される
  3. 接続エッジも自動削除

4. PropertyPanelが閉じる

3.9 エッジ削除

3.9.1 処理フロー

```
// 削除ボタンクリック
onClick={() => setDeleteEdgeConfirmOpen(true)}

// 削除確認モーダルのOKボタン
onOk={() => {
  if (onDeleteEdge) {
    onDeleteEdge(selectedEdge.id);
  }
  setDeleteEdgeConfirmOpen(false);
  onClose();
}}
```

3.9.2 UI動作

- ユーザー操作: "エッジを削除"ボタンをクリック
- モーダル表示: "エッジ削除の確認"
- 確認: "削除"ボタンをクリック
- 結果:
  1. 親コンポーネントのhandleDeleteEdge実行
  2. エッジが削除される
  3. PropertyPanelが閉じる

---

4. エラー表示機能

4.1 renderErrors関数

```
const renderErrors = (errors?: ErrorContent[]) => {
  if (!errors || errors.length === 0) return null;

  return (
    <Card
      size="small"
      style={{
        marginBottom: 16,
        backgroundColor: '#fff3e0',
        border: '1px solid #ff9800'
      }}
    >
      <Space orientation="vertical" style={{ width: '100%' }}>
        <Text strong style={{ color: '#ff9800' }}>
          <WarningOutlined /> {errors.length}件の警告
        </Text>
        {errors.map((error, idx) => (
          <div key={idx} style={{ paddingLeft: 8, borderLeft: '2px solid #ff9800' }}>
            <Text strong style={{ fontSize: 12 }}>{error.type}</Text>
            <br />
            <Text type="secondary" style={{ fontSize: 11 }}>{error.message}</Text>
            {error.details && (
              <>
                <br />
                <Text type="secondary" style={{ fontSize: 10 }}>
                  詳細: {String(error.details)}
                </Text>
              </>
            )}
          </div>
        ))}
      </Space>
    </Card>
  );
};
```

表示内容:

- エラー件数
- 各エラーの詳細:
  - type（エラー種別）
  - message（エラーメッセージ）
  - details（詳細情報、あれば）

5. スタイル仕様

5.1 パネルコンテナ

```
style={{
  position: 'absolute',
  right: 0,
  top: 0,
  width: 350,
  height: '100%',
  backgroundColor: '#fff',
  borderLeft: '1px solid #f0f0f0',
  display: 'flex',
  flexDirection: 'column',
  zIndex: 1000,
  boxShadow: '-2px 0 8px rgba(0,0,0,0.1)',
}}
```

説明:

- 位置: 右側固定（absolute）
- 幅: 350px
- 高さ: 100%
- z-index: 1000（キャンバスより前面）
- 影: 左側に影を表示

5.2 ヘッダー

```
style={{
  padding: '12px 16px',
  borderBottom: '1px solid #f0f0f0',
  display: 'flex',
  justifyContent: 'space-between',
  alignItems: 'center',
  backgroundColor: '#1890ff',
```

```
  color: '#fff',
}}
```

5.3 編集エリア

```
style={{ flex: 1, overflow: 'auto', padding: 16 }}
```

---

6. エラーハンドリング

6.1 エラーケース

エラーケース	対処方法
ノードIDが空	message.error表示、更新中止
ノードIDが重複	message.error表示、更新中止
バリデーションエラー	エラーカードに警告表示

6.2 バリデーションエラー表示

- selectedNode.data.errorsまたはselectedEdge.data.errorsに含まれるエラーを表示
- オレンジ色のカードで警告表示
- 各エラーの詳細（type、message、details）を表示

---

7. 実装ファイル

7.1 ファイルパス

- **メインファイル**: `frontend/src/components/editorv2/components/PropertyPanel.tsx`

7.2 関連ファイル

- **型定義**:
  - `frontend/src/types/editor.ts` (NodeData, EdgeData, MethodData)
  - `frontend/src/types/maiml-unified.ts` (Template, Reference)
  - `frontend/src/types/maimlValidation.ts` (ErrorContent)
- **コンテキスト**: `frontend/src/contexts/TemplateContext.tsx`

7.3 依存コンポーネント

- Ant Design:
  - Input, TextArea
  - Typography
  - Button
  - Divider
  - Modal
  - Select
  - Checkbox
  - Space
  - Card
  - message
- Ant Design Icons:
  - CloseOutlined
  - DeleteOutlined
  - WarningOutlined
- React Flow:
  - Node
  - Edge

7.4 親コンポーネント

- PetriNetEditorInner

---

8. 変更履歴

日付	変更内容	担当者
2026-03-12	実装ベースで仕様書を新規作成	Cline
2026-02-16	テンプレート編集機能を追加	Development Team